Testing & Quality Assurance

Project: Job Board API
GitHub Repository: https://github.com/Taweesaksangkorane/jobboard-api

1. Testing Objectives and Scope

The purpose of Testing and Quality Assurance is to verify the correctness, reliability, and security of the Job Board API system. Testing focuses on important system features, including user authentication, role-based authorization, job management, application status updates, notifications, input validation, and API security.

Two testing approaches are used:

1. Automated Testing: Uses test scripts to verify API functionality, authentication, authorization, input validation, and security.
2. Manual Testing: Tests the system through a web browser and API testing tools to verify functionality from the user's perspective.

2. Automated Testing

2.1 Testing Overview

The Job Board API project includes automated backend API tests using the Node.js Test Runner.

The test script is located in tests/worker.test.mjs and contains 12 test cases covering API functionality, input validation, authentication, authorization, and security.

2.2 Automated Test Cases

Test ID	Test Case	Expected Result
AT-01	Health Check	Returns HTTP 200 with security headers
AT-02	CORS Protection	Rejects unauthorized origins
AT-03	Authentication	Unauthenticated users cannot create job postings
AT-04	Authorization	Students cannot create job postings
AT-05	Signed Session	Valid sessions are successfully verified

AT-06	Role-based Session	Student and Employer roles are correctly separated
AT-07	Job Input Validation	Rejects incomplete job posting data
AT-08	Phone Validation	Rejects invalid phone numbers
AT-09	Webhook Authentication	Rejects invalid webhook secrets
AT-10	Webhook Validation	Rejects incomplete webhook payloads
AT-11	SQL Parameter Binding	Uses parameter binding for job searches
AT-12	Search Validation	Rejects search keywords exceeding the limit

2.3 Automated Testing Evidence

Figure 1: Automated Test Execution Results

Objective: To verify that the backend API functions correctly and passes the predefined automated tests.

Testing Procedure:

1. Open the jobboard-api project in Visual Studio Code.
2. Open the Terminal.
3. Install dependencies if necessary by running npm install.
4. Execute the following command:

```
None
npm run test:worker
```

5. Review the total number of tests, passed tests, and failed tests.

Expected Result: All predefined test cases should pass without errors.

Screenshot Instructions: Capture the Terminal displaying the test command and the final test results, including the total number of tests, passed tests, and failed tests.

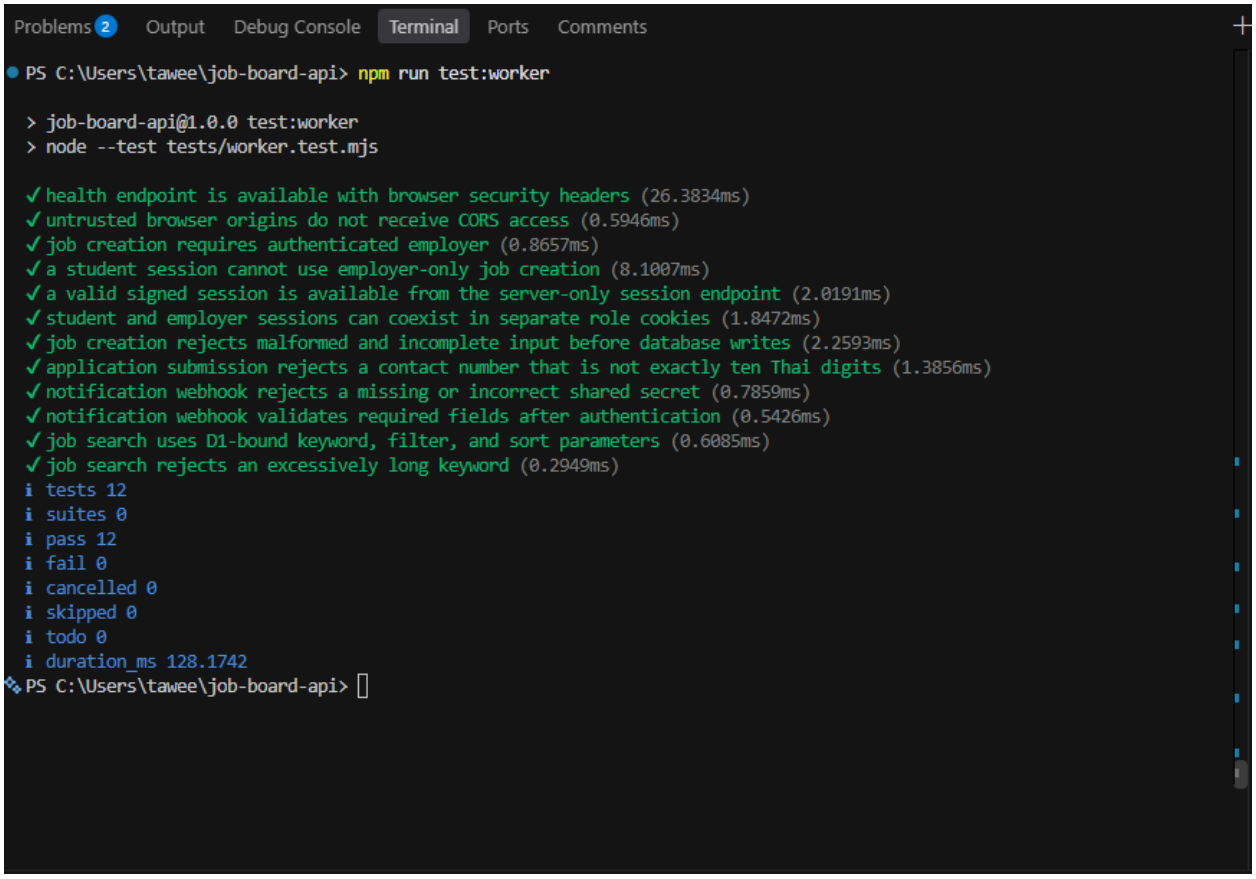


Figure 1. Automated backend API testing results showing the execution status of predefined test cases.

Actual Result: All 12 automated test cases passed successfully with 0 failures. The total execution time was 128.101 ms.
Test Status: ☑ PASS ☐ FAIL ☐ NOT TESTED

Figure 2: Overall Quality Check Results

Objective: To verify overall project quality, including automated tests, code validation, frontend build, and deployment readiness.

Testing Procedure:

1. Open the Terminal in the project directory.
2. Execute the following command:

```
None
npm test
```

3. Review the results of Worker Tests, Frontend Lint, Frontend Build, and Wrangler Deployment Dry-run.

Expected Result: All configured quality checks should complete successfully without errors.

Screenshot Instructions: Capture the Terminal displaying the completed quality checks and their results. Include any error messages if a check fails.



Figure 2. Overall project quality verification results, including automated testing, code validation, frontend build, and deployment checks.

Actual Result:Worker tests, frontend TypeScript check, frontend build, and Wrangler deployment dry-run completed successfully.

Test Status: ☑ PASS ☐ FAIL ☐ NOT TESTED

3. Manual Testing

3.1 Testing Overview

Manual Testing is performed to verify the functionality of the Job Board system from the user's perspective.

The testing process simulates real user interactions through a web browser and API testing tools. It focuses on website accessibility, student authentication, employer application management, notifications, input validation, and webhook security.

3.2 Manual Test Cases

Test ID	Test Case	Expected Result
M-01	Website Accessibility	Website loads successfully over HTTPS
M-02	Student Login	Student can log in successfully
M-03	Application Status Update	Employer can update application status
M-04	Student Notification	Student receives the updated status notification
M-05	Phone Number Validation	System rejects invalid phone numbers
M-06	Duplicate Webhook Event	System prevents duplicate notifications
M-07	Invalid Webhook Secret	API returns HTTP 401 Unauthorized

3.3 Manual Testing Evidence

Figure 3: Website Accessibility Test (M-01)

Objective: To verify that users can successfully access the Job Board website.

Testing Procedure:

1. Open Google Chrome or Microsoft Edge.

2. Navigate to https://team18-job-board-api.tawee-dev.workers.dev/.
3. Verify that the website loads correctly.

Expected Result: The website should load successfully and display the main interface without errors.

Screenshot Instructions: Capture the Job Board homepage, including the website URL in the browser's address bar.

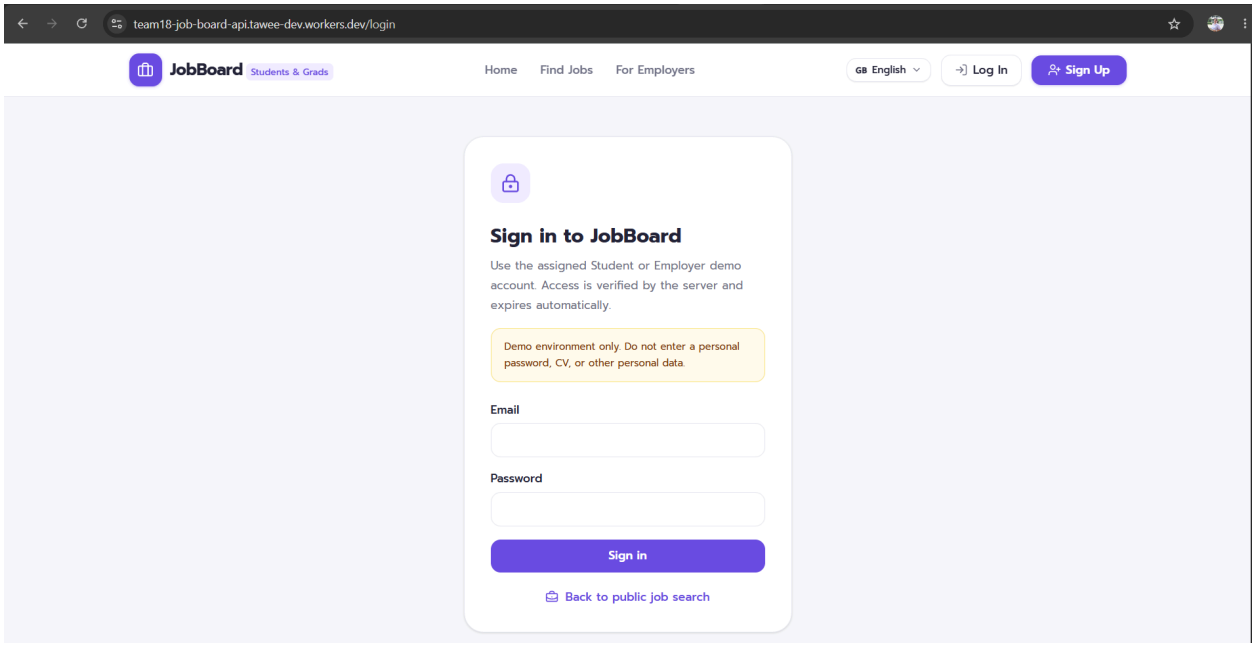


Figure 3. Website accessibility test showing the successful loading of the Job Board homepage.

Actual Result: The Job Board login page loaded successfully over HTTPS without any visible errors.
Test Status: ☑ PASS ☐ FAIL ☐ NOT TESTED

Figure 4: Student Login Test (M-02)

Objective: To verify that students can log in and access features assigned to their role.

Testing Procedure:

1. Open the Job Board website.
2. Navigate to the Login page.
3. Log in using a test Student account.
4. Open the Dashboard or Notifications page.

Expected Result: The student should successfully log in and access the authorized features.

Screenshot Instructions: Capture the Student Dashboard or Notifications page after successfully logging in. Do not expose account passwords.

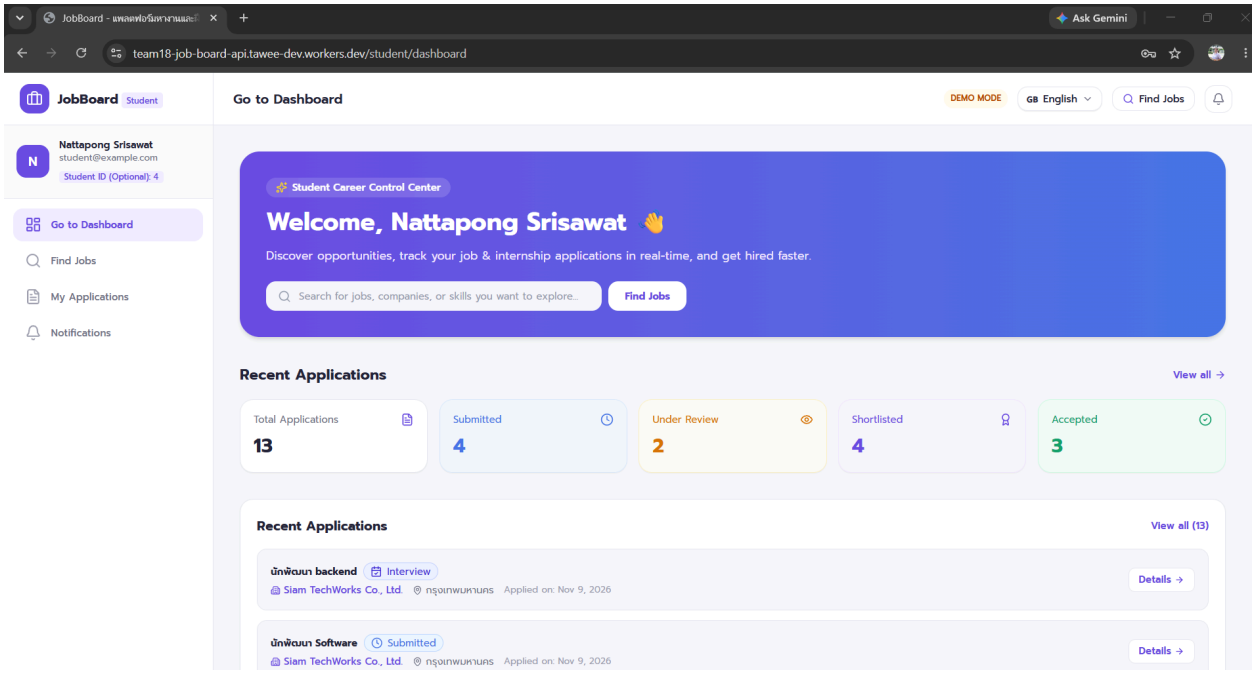


Figure 4. Student authentication test demonstrating successful access to role-specific features.

Actual Result: The Student Dashboard was successfully displayed with student information, recent applications, and role-specific navigation menus.
Test Status: ☑ PASS ☐ FAIL ☐ NOT TESTED

Figure 5: Application Status Update Test (M-03)

Objective: To verify that employers can successfully update student application statuses.

Testing Procedure:

1. Log in using an Employer account.
2. Navigate to the job applications management page.
3. Select a test application.
4. Change its status, for example, from shortlisted to interview.
5. Verify that the updated status is saved and displayed correctly.

Expected Result: The system should successfully update and display the new application status.

Screenshot Instructions: Capture the Employer application management page displaying the updated application status.

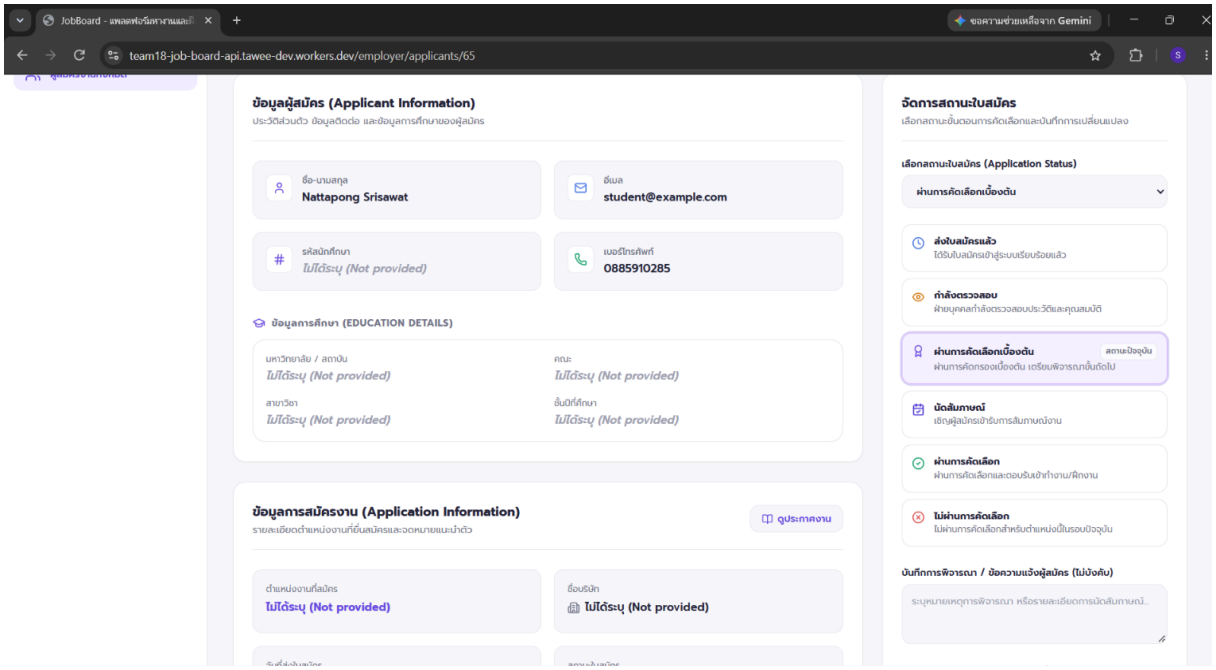


Figure 5. Application status management test showing an Employer successfully updating a student's job application status.

Actual Result: The system successfully displays applicant information, allows employers to update application statuses, and automatically notifies students when their application status changes.
Test Status: ☑ PASS ☐ FAIL ☐ NOT TESTED

Figure 6: Student Notification Test (M-04)

Objective: To verify that students receive notifications when employers update their application statuses.

Testing Procedure:

1. Complete the application status update test (M-03).
2. Log in using the Student account associated with the application.
3. Navigate to the Notifications page.
4. Check whether a notification related to the updated application status appears.

Expected Result: The student should receive a notification corresponding to the updated application status.

Screenshot Instructions: Capture the Notifications page displaying the latest application status notification.

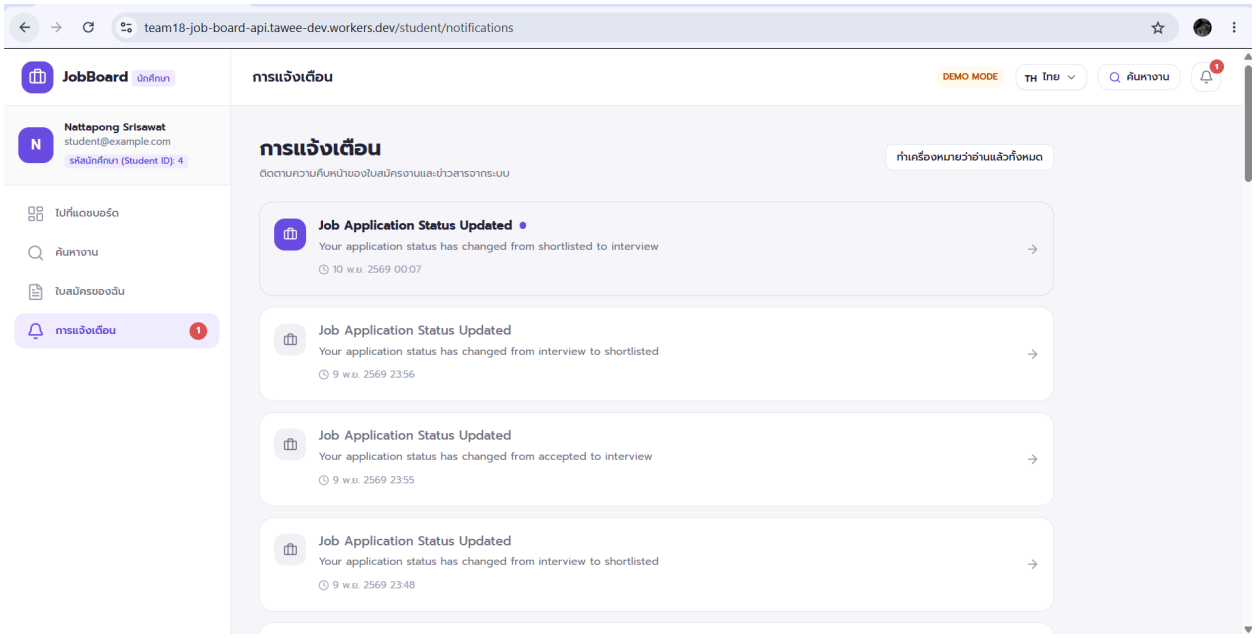


Figure 6. Student notification test showing a notification generated after an Employer updates an application status.

Actual Result: The student successfully received a notification after the Employer updated the application status. The notification displayed the correct status change from Shortlisted to Interview.

Test Status: ☑ PASS ☐ FAIL ☐ NOT TESTED

Figure 7: Phone Number Validation Test (M-05)

Objective: To verify that the system correctly validates phone numbers and rejects invalid input.

Testing Procedure:

1. Open a form containing a phone number field.
2. Enter a phone number containing more than 10 digits.
3. Submit the form.
4. Observe the system response.

Expected Result: The system should reject the invalid phone number and display a validation error or return HTTP 400.

Screenshot Instructions: Capture the form showing the invalid phone number and the corresponding validation error.

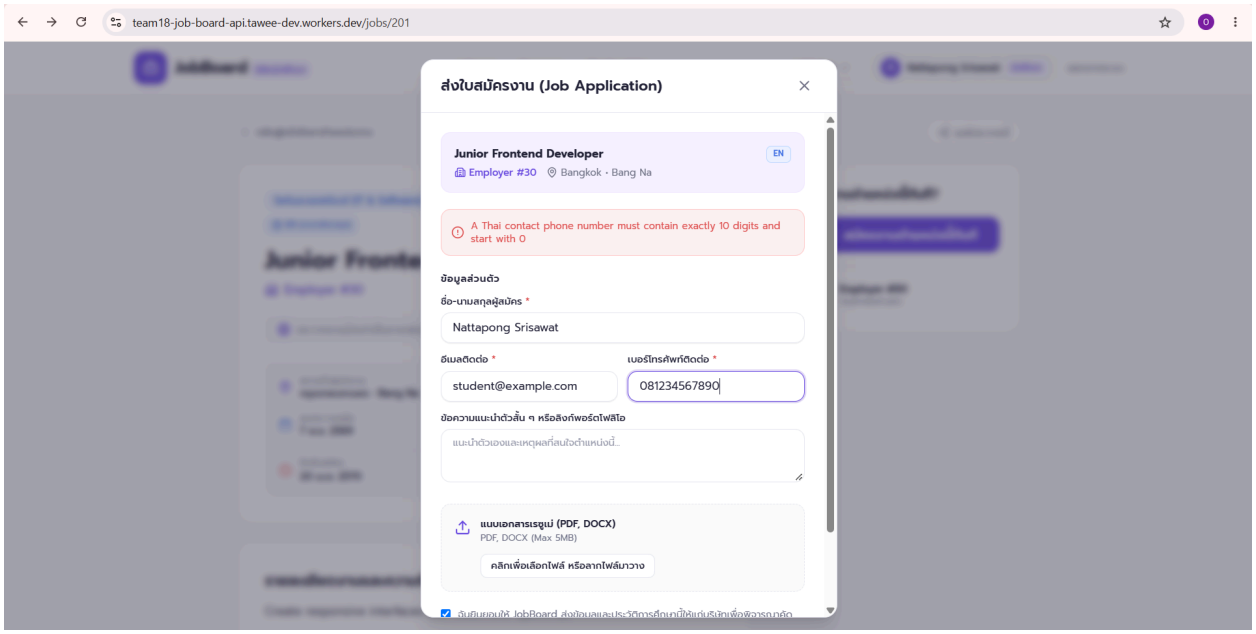


Figure 7. Phone number validation test demonstrating the system's handling of invalid user input.

Actual Result: The system detected that the phone number contained more than 10 digits and displayed a validation error message indicating that a Thai phone number must contain exactly 10 digits and start with 0.
Test Status: ☑ PASS ☐ FAIL ☐ NOT TESTED

Figure 8: Webhook Authentication Test (M-07)

Objective: To verify that the API rejects unauthorized webhook requests.

Testing Procedure:

1. Open Postman.
2. Select the webhook API request from the project's Postman Collection.
3. Configure an invalid webhook secret.
4. Send the HTTP request.
5. Verify the returned HTTP status code.

Expected Result: The API should reject the request and return HTTP 401 Unauthorized.

Screenshot Instructions: Capture the Postman response showing HTTP 401 Unauthorized and the response body. Hide all secrets and authentication tokens.

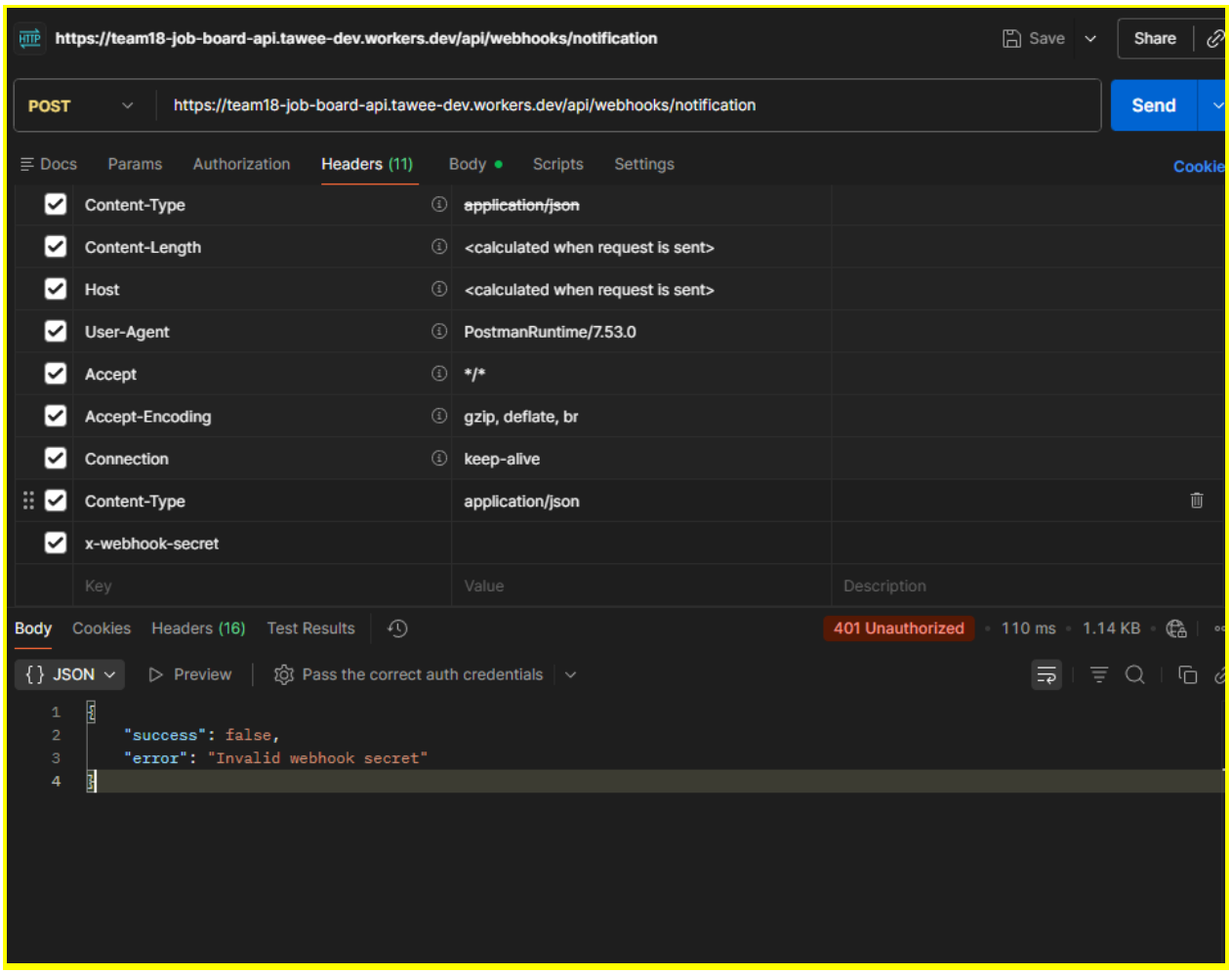


Figure 8. Webhook authentication test demonstrating that unauthorized API requests are rejected.

Actual Result: The API rejected the request with HTTP 401 Unauthorized and returned "Invalid webhook secret".

Test Status: ☑ PASS ☐ FAIL ☐ NOT TESTED

Figure 9: Duplicate Webhook Event Test (M-06)

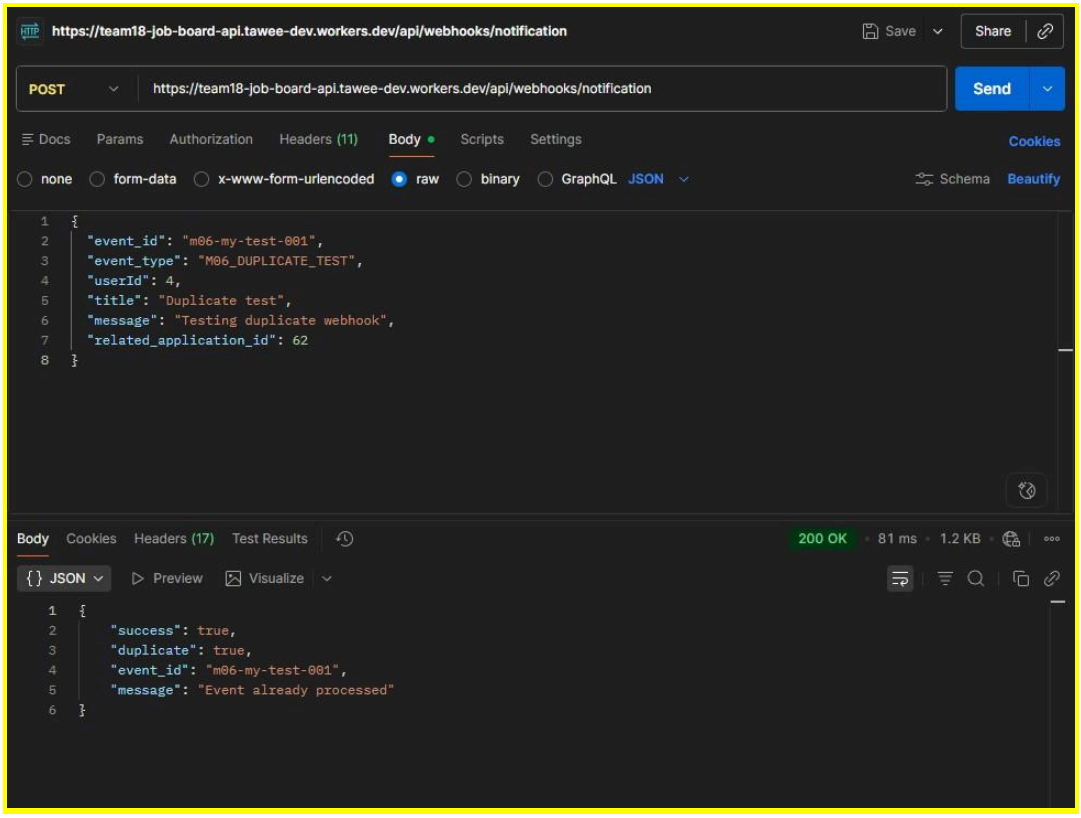
Objective: To verify that duplicate webhook events do not create duplicate notifications.

Testing Procedure:

1. Open Postman or the configured webhook testing tool.
2. Prepare a valid webhook event payload using a test event identifier.
3. Send the same webhook event twice.
4. Check the generated notifications.
5. Verify that duplicate notifications are not created.

Expected Result: The system should process duplicate events safely and prevent duplicate notifications.

Screenshot Instructions: Capture the webhook requests showing the same event identifier and the resulting notifications list. If this test cannot be completed, record the status as NOT TESTED.



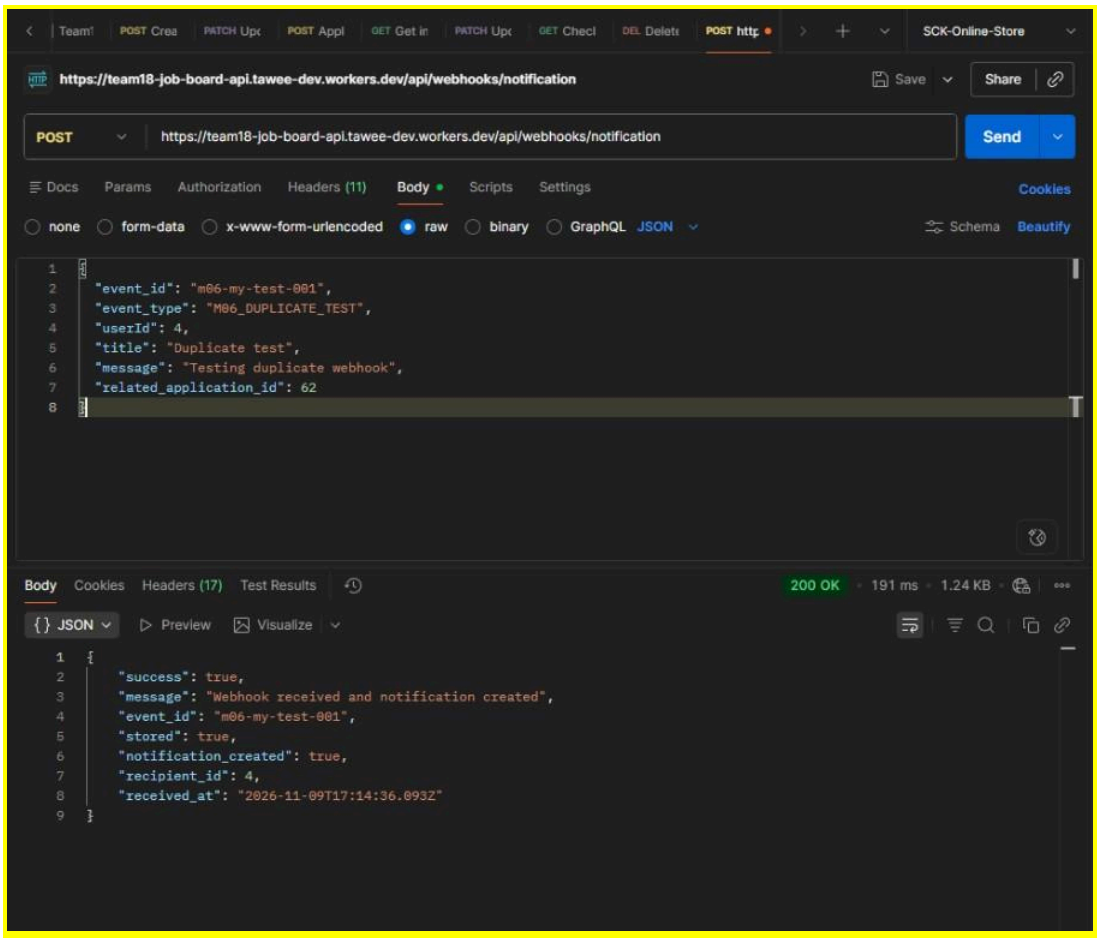


Figure 9. Duplicate webhook event test verifying whether repeated events generate duplicate notifications.

Actual Result: The first webhook request created a notification with stored: true. Replaying the same event returned duplicate: true and did not create another notification.

Test Status: ☑ PASS ☐ FAIL ☐ NOT TESTED

4. Testing Results Summary

The testing process covers important functional and security requirements of the Job Board API. Automated Testing focuses on verifying backend API behavior, while Manual Testing evaluates real user interactions and system responses.

4.1 Test Results

Testing Type	Total Test Cases	Passed	Failed	Not Tested
Automated Testing	12	12	0	0
Manual Testing	7	7	0	0
Total	19	19	0	0

4.2 Failed and Pending Test Cases

Any test case that does not meet its expected result must be recorded as FAILED, with a description of the observed issue and supporting evidence.

Test cases that cannot be executed due to missing test data, external dependencies, or incomplete integration should be recorded as NOT TESTED or PENDING rather than FAILED.

M-06 was verified successfully. The first request created a notification, and replaying the same event returned duplicate: true without creating another notification.

5. Conclusion

The Job Board API successfully passed all 19 test cases, consisting of 12 automated tests and 7 manual tests, with no failures.

The results demonstrate that the system's key functionalities, authentication, role-based authorization, input validation, webhook integration, and security controls operate as expected within the academic demonstration environment.

Testing evidence, including screenshots and API responses, has been collected to support the final evaluation.